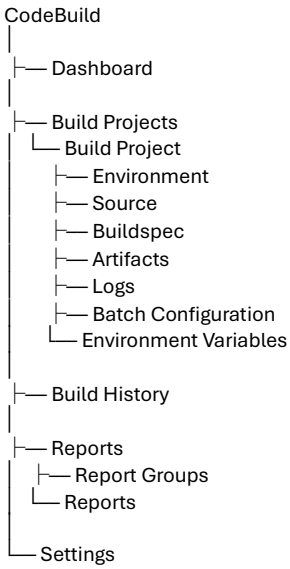
AWS CodeBuild — Console Dashboard Wise Important Interview Questions & Answers



1. What is the CodeBuild Dashboard?
Answer:
CodeBuild Dashboard is the main console page where we get an overview of our build activity.
It helps us see build projects, recent builds, and overall build status.
From here, we can quickly navigate to projects and build history.
It is mainly used for monitoring and managing CodeBuild.
For example:
Suppose our application build is failing frequently.
We can open the CodeBuild Dashboard and check the recent build status.
From there, we can open the failed build and check its logs.
Diagram:
CodeBuild
↓
Dashboard
↓
Build Status
↓
Build Projects / History
Simple ga gurthupettuko:
Dashboard = Overview of CodeBuild

Build Projects
2. What is a CodeBuild Project?
Answer:
A CodeBuild Project contains the configuration required to run an application build.
It defines the source, build environment, build commands, artifacts, logs, and other settings.
Whenever we start a build, CodeBuild uses this project configuration.
We can create separate projects for different applications.
For example:
Suppose we have a Java application called ShoppingApp.
We can create a CodeBuild project containing its GitHub source, Java environment, build commands, and artifact settings.
Whenever we start the project, CodeBuild follows that configuration.
Diagram:
CodeBuild Project
↓
Source
↓
Environment
↓
Build
↓
Artifact
Simple ga gurthupettuko:
Build Project = Complete build configuration

Environment
3. What is the Environment in a CodeBuild Project?

Answer:
The Environment defines the compute environment where the build actually runs.
It includes the operating system, compute type, runtime image, and other build requirements.
CodeBuild creates the required environment when the build starts.
The build commands are executed inside this environment.
For example:
Suppose our application is written in Java.
We can select a Linux build environment with the required Java runtime.
CodeBuild uses that environment to compile and test the application.
Diagram:
CodeBuild
↓
Build Environment
↓
Linux + Java
↓
Compile + Test
Simple ga gurthupettuko:
Environment = Where build runs

---

4. What is Compute Type in CodeBuild Environment?
Answer:
Compute Type determines the compute resources available to the build environment.
We select an appropriate compute size based on the application's build requirements.
Small builds may need fewer resources, while large builds may require more CPU and memory.
Choosing the right compute type helps balance build performance and cost.
For example:
A small Python project may need a small build environment.
A large application with many dependencies may require more CPU and memory.
So we select the compute type according to the build workload.
Diagram:
Build
↓
Compute Type
↓
CPU + Memory
↓
Build Performance
Simple ga gurthupettuko:
Compute Type = Build machine size

Source
5. What is Source in CodeBuild?
Answer:
Source defines where CodeBuild gets the application source code.
We configure a supported source provider such as GitHub, Bitbucket, CodeCommit, or S3.
When a build starts, CodeBuild retrieves the required source code.
That source is then passed to the build environment.
For example:
Suppose our application code is stored in GitHub.
We configure the GitHub repository as the source for the CodeBuild project.
When the build starts, CodeBuild downloads the required code.
Diagram:
GitHub
↓
CodeBuild
↓
Download Source
↓
Build
Simple ga gurthupettuko:
Source = Where code comes from

Buildspec
6. What is Buildspec in CodeBuild?
Answer:
Buildspec contains the instructions that CodeBuild follows during the build process.
It can define phases such as install, pre_build, build, and post_build.
It can also define artifacts and other build-related configuration.
The buildspec is commonly stored as buildspec.yml with the source code.

For example:
Suppose we have a Java application.
The buildspec can tell CodeBuild to install dependencies, run Maven commands, execute tests, and create a JAR file.
CodeBuild follows these instructions during the build.
Diagram:
Source Code
↓
buildspec.yml
↓
Build Commands
↓
Test
↓
Artifact
Simple ga gurthupettuko:
Buildspec = Instructions for the build

Artifacts
7. What are Artifacts in CodeBuild?
Answer:
Artifacts are the output files generated by a successful build.
They can include JAR files, ZIP files, application packages, or other build outputs.
CodeBuild can store the generated artifacts in a configured destination such as S3.
These artifacts can then be used by deployment services.
For example:
Suppose CodeBuild compiles a Java application and creates app.jar.
CodeBuild can package that file as an artifact.
The artifact can then be stored in S3 and used for deployment.
Diagram:
Source
↓
CodeBuild
↓
Build
↓
app.jar
↓
S3 Artifact
Simple ga gurthupettuko:
Artifact = Build output

Logs
8. What are Logs in CodeBuild?
Answer:
CodeBuild logs show the commands and output generated during a build.
They help us understand whether each build phase succeeded or failed.
Logs can be sent to CloudWatch Logs and/or S3 depending on the project configuration.
They are very useful for troubleshooting build failures.
For example:
Suppose our build fails because a dependency is missing.
We open the CodeBuild logs and check the failed command and error message.
Based on the error, we can correct the build configuration.
Diagram:
CodeBuild
↓
Build
↓
Error
↓
Logs
↓
Troubleshooting
Simple ga gurthupettuko:
Logs = Find build errors

Batch Configuration
9. What is Batch Configuration in CodeBuild?
Answer:
Batch Configuration allows CodeBuild to run multiple builds together as a batch.
It is useful when we need to build multiple configurations or components at the same time.

Batch builds can be configured based on the application's requirements.
This can help reduce the time required for multiple related builds.
For example:
Suppose an application has separate frontend and backend builds.
Instead of handling each build completely separately, we can use batch configuration when the workload supports it.
CodeBuild can process the configured builds as a batch.
Diagram:
CodeBuild
↓
Batch Build
├— Frontend
├— Backend
└— Tests
Simple ga gurthupettuko:
Batch Configuration = Run multiple builds together

Environment Variables
10. What are Environment Variables in CodeBuild?
Answer:
Environment variables provide configuration values that are available to the build process.
They can contain values such as application environment, build configuration, or other non-secret settings.
CodeBuild makes these variables available to the build commands.
Sensitive values should be handled securely using appropriate AWS services rather than exposing secrets directly.
For example:
Suppose our build needs to know whether it is building for dev or prod.
We can provide an environment variable such as ENVIRONMENT=prod.
The build commands can then use that value.
Diagram:
CodeBuild
↓
Environment Variables
↓
ENVIRONMENT=prod
↓
Build Commands
Simple ga gurthupettuko:
Environment Variable = Value available during build

Build History
11. What is Build History in CodeBuild?
Answer:
Build History shows the builds that have been executed for CodeBuild projects.
It provides information such as build status, start time, duration, and other build details.
We can open a particular build to investigate its execution.
It is mainly useful for monitoring and troubleshooting previous builds.
For example:
Suppose today's deployment failed.
We can open Build History and compare today's failed build with yesterday's successful build.
This can help identify what changed.
Diagram:
Build History
↓

Build 1 → Success
Build 2 → Success
Build 3 → Failed

↓
Open Build Details
Simple ga gurthupettuko:
Build History = Previous build records

Reports
12. What are Reports in CodeBuild?
Answer:
CodeBuild Reports provide information about test results generated during builds.
They can show information such as test cases, passed tests, failed tests, and test execution results.
Reports help developers understand the quality and test status of an application build.
They are especially useful when automated tests are part of the CI process.
For example:
Suppose our build runs 100 unit tests.

After the build, the report can show how many tests passed and how many failed.
Developers can use that information to troubleshoot application issues.
Diagram:
CodeBuild
↓
Run Tests
↓
Test Results
↓
CodeBuild Reports
Simple ga gurthupettuko:
Reports = Build test results

Report Groups
13. What is a Report Group in CodeBuild?
Answer:
A Report Group is used to organize related test reports in CodeBuild.
It defines where reports belonging to a particular testing purpose are grouped.
Multiple build executions can generate reports associated with the same report group.
This makes test-result management easier.
For example:
Suppose our application runs unit tests on every build.
We can have a report group for those unit-test results.
Each build can generate a new report under that group.
Diagram:
Report Group
↓

Build 1 Test Report
Build 2 Test Report
Build 3 Test Report

Simple ga gurthupettuko:
Report Group = Container for test reports

Settings
14. What is the Settings section in CodeBuild?
Answer:
The CodeBuild Settings area contains configuration related to managing the CodeBuild service and projects.
Depending on the console and account configuration, it can contain service-level or related configuration options.
These settings help administrators manage how CodeBuild integrates with other AWS resources.
We should review the settings when configuring CodeBuild for a production environment.
For example:
Suppose our organization needs CodeBuild projects to work with specific AWS resources and permissions.
We review the relevant settings and IAM permissions before running production builds.
This helps ensure the build process has the required access.
Diagram:
CodeBuild
↓
Settings
↓
Configuration
↓
Build Integration
Simple ga gurthupettuko:
Settings = CodeBuild configuration